

CCES – College of Computer Engineering and Sciences

CS3401 – Algorithm Design and Analysis

Chapter 4- Decrease & Conquer

References:

- ❖ *Introduction to the Design and Analysis of Algorithms. Anany V. Levitin, Pearson 3rd Edition.*

Decrease & Conquer

Introduction

The ***decrease-and-conquer*** technique is based on exploiting the relationship between a solution to a given instance of a problem and a solution to its smaller instance. Consists in 3 phases:

1. Reduce problem instance to smaller instance of the same problem
2. Solve smaller instance
3. Extend solution of smaller instance to obtain solution to original instance

Can be implemented either top-down (***inductive approach***) : recursive approach starting from the original problem to the smallest instance or bottom-up (***incremental approach***): iterative approach starting with a solution to the smallest instance of the problem.

- There are three major variations of decrease-and-conquer:
 - decrease by a constant
 - decrease by a constant factor
 - variable size decrease

3 Types of Decrease and Conquer

1. Decrease by a constant (usually by 1):
 - insertion sort
 - topological sorting
 - algorithms for generating permutations, subsets
2. Decrease by a constant factor (usually by half)
 - binary search and bisection method
 - exponentiation by squaring
 - multiplication à la russe
3. Variable-size decrease
 - Euclid's algorithm
 - selection by partition
 - Nim-like games

The 3 types of Decrease and Conquer

- In the coming slides, examples will be given regarding the three types of the decrease and conquer techniques, which are:
 1. Decrease by Constant
- 2. Decrease by Constant factor
 3. Variable Size Decrease

Decrease & Conquer

Decrease by a constant (or Decrease by one)

The size of an instance is reduced by the same constant (usually 1) on each iteration.

computing a^n where $a > 0$ and n is a nonnegative integer



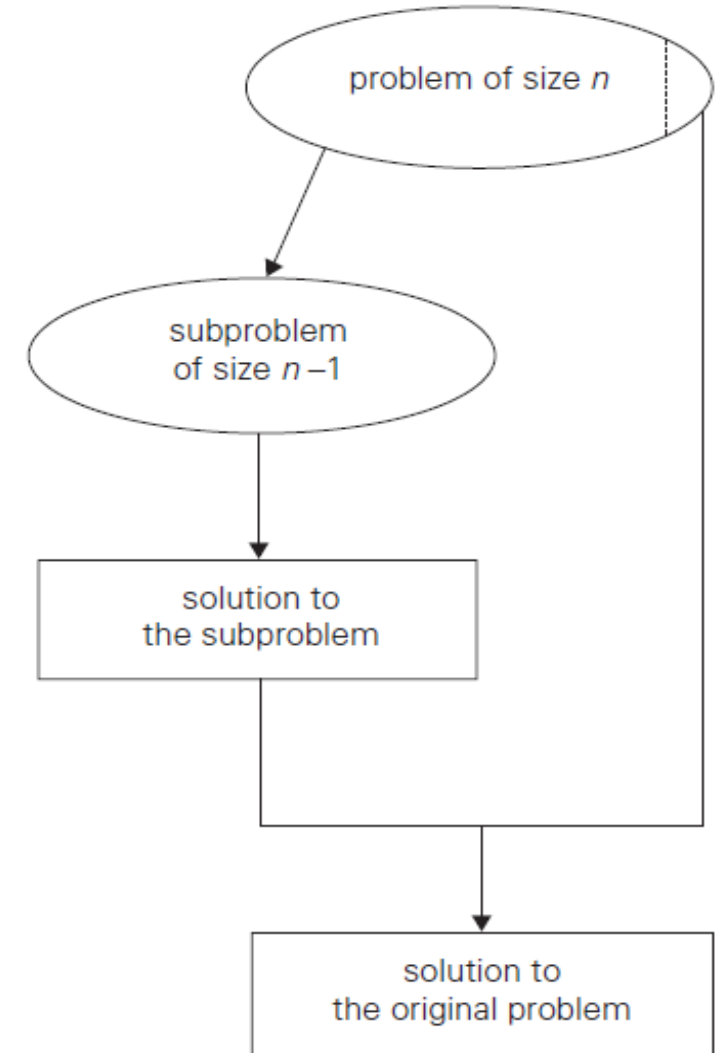
$$a^n = a^{n-1} \times a$$

Top down: Recursive

$$f(n) = \begin{cases} f(n-1) \cdot a & \text{if } n > 0, \\ 1 & \text{if } n = 0, \end{cases}$$

Bottom up: iterative

Multiply 1 by a , n times
The same as the brute-force algorithm



Decrease & Conquer

Decrease by a constant factor

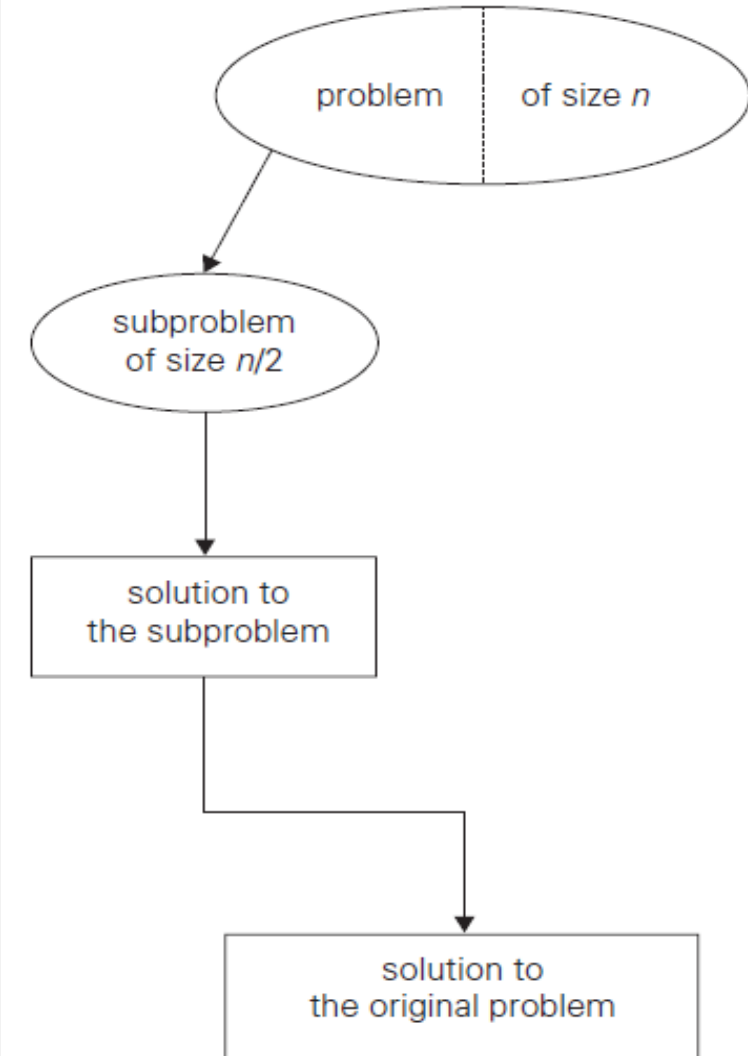
Reducing a problem instance by the same constant factor on each iteration of the algorithm (mostly 1/2).

computing a^n where $a > 0$ and n is a nonnegative integer



$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive,} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n \text{ is odd,} \\ 1 & \text{if } n = 0. \end{cases}$$

$a^{16} = (((a*a)^2)^2)^2 \rightarrow 4$ multiplications
 $a^{32} = (((((a*a)^2)^2)^2)^2)^2 \rightarrow 5$ multiplications
...
 $a^{1024} \rightarrow 10$ multiplications
Obviously this is an $O(\log n)$ algorithm



Decrease & Conquer

Variable size decrease

The size-reduction pattern varies from one iteration of an algorithm to another.

Examples:

- Euclid's Algorithm to compute the greatest common divisor

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

Ex.: $\gcd(80, 44) = \gcd(44, 36) = \gcd(36, 12) = \gcd(12, 0) = 12$

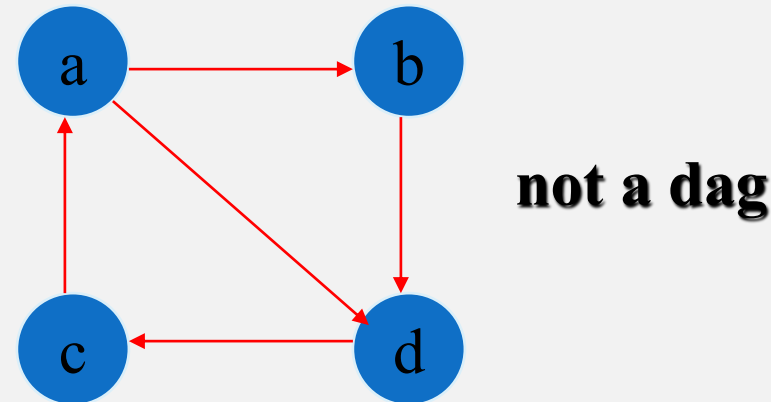
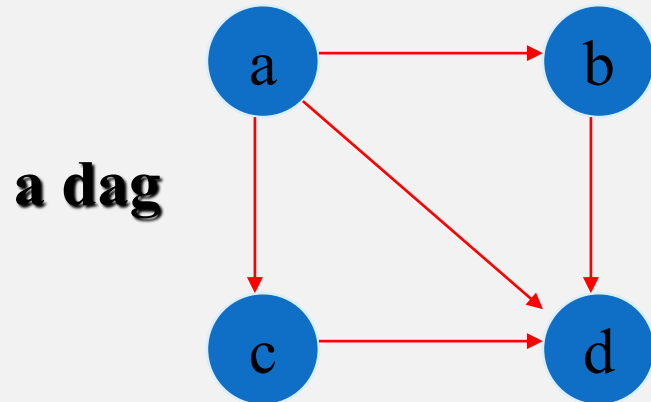
One can prove that the size, measured by the second number, decreases at least by half after two consecutive iterations.

Hence, $T(n) \in O(\log n)$

Decrease by Constant (or Decrease by One)

Topological Sorting

- Ordering of the vertices of a directed graph such that for every edge uv , u comes before v in the ordering.
- directed acyclic graph (dag): directed graph with no (directed) cycles.



Arise in modeling many problems that involve prerequisite constraints (construction projects, document version control)

Vertices of a dag can be linearly ordered so that for every edge its starting vertex is listed before its ending vertex (topological sorting). Being a dag is also a necessary condition for topological sorting be possible.

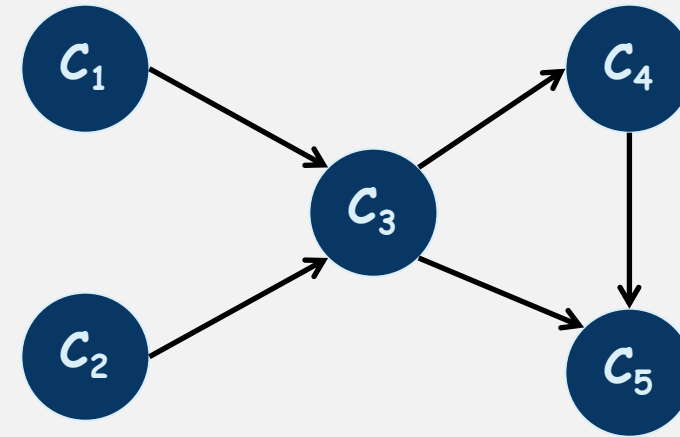
Decrease by Constant (or Decrease by One)

Topological Sorting

- Set of 5 courses: $\{ C_1, C_2, C_3, C_4, C_5 \}$
- C_1 and C_2 have no prerequisites
- C_3 requires C_1 and C_2
- C_4 requires C_3
- C_5 requires C_3 and C_4

In which order should the student take the courses?

- Using depth-first search (Traversal stack)
- Using of Adjacency matrix
- Using source-removal algorithm



Digraph representation

Decrease by Constant (or Decrease by One)

Topological Sorting

In which order should the student take the courses?

- Using depth-first search

DFS traversal stack

C₂_{5,5}
C₅_{4,1}
C₄_{3,2}
C₃_{2,3}
C₁_{1,4}

Push order

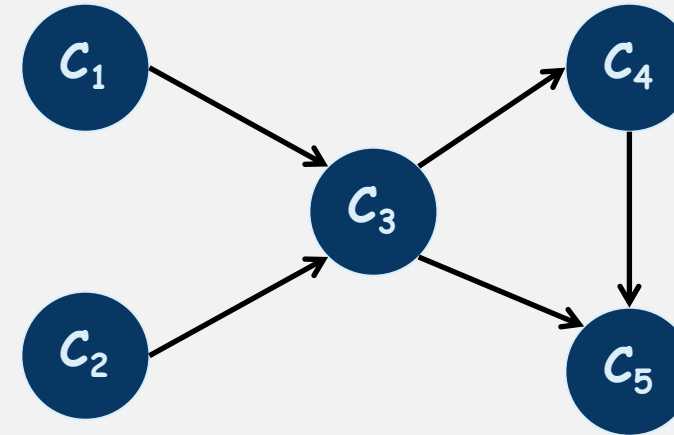
Pop order

The popping-off order:

C₅, C₄, C₃, C₁, C₂

The topologically sorted list:

C₂ C₁ → C₃ → C₄ → C₅



Digraph representation

If a back edge has been encountered, the digraph is not a dag, and topological sorting of its vertices is impossible!!!

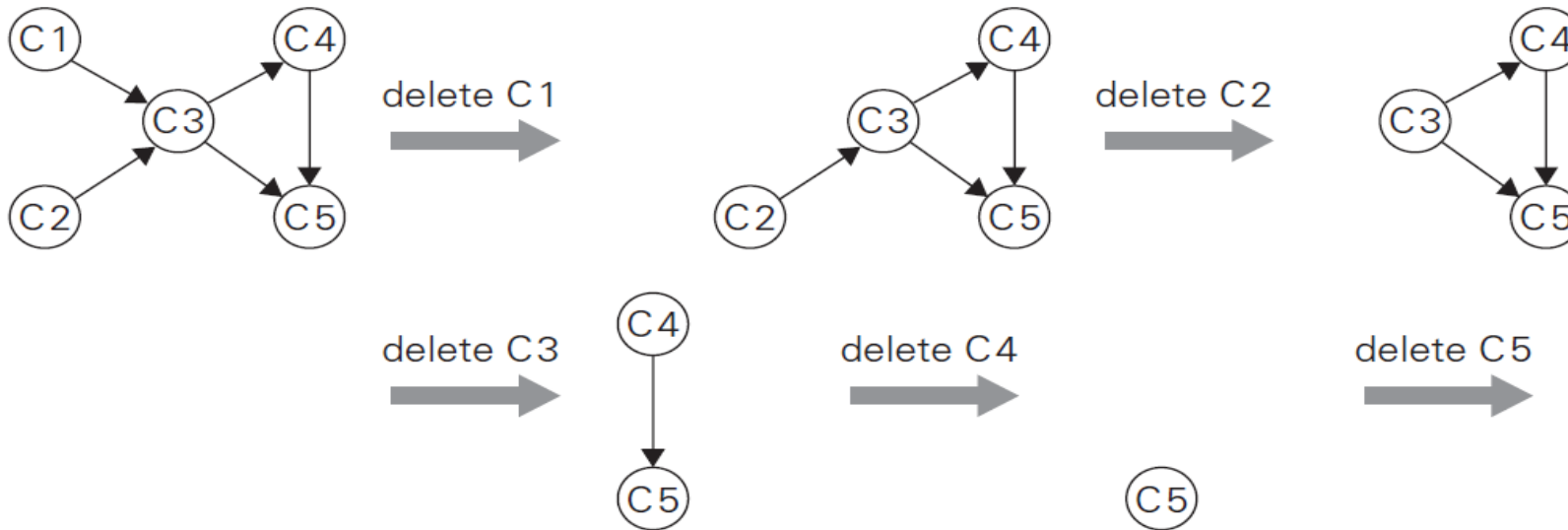
Decrease by Constant (or Decrease by One)

Topological Sorting

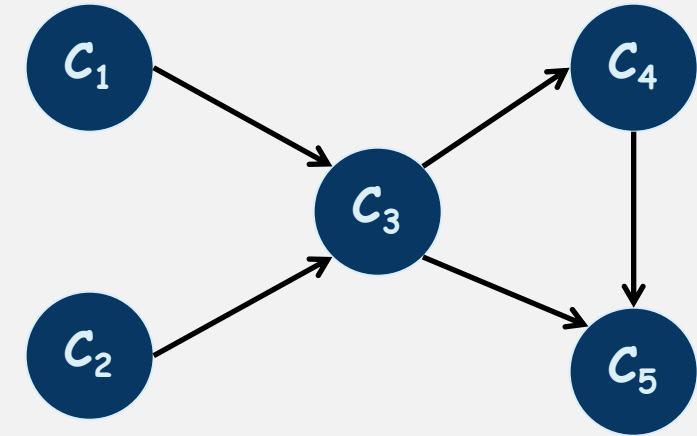
In which order should the student take the courses?

- Using source-removal algorithm

Repeatedly identify and remove a *source* (a vertex with no incoming edges) and all the edges incident to it until either no vertex is left (problem is solved) or there is no source among remaining vertices (not a dag)



The solution obtained is C1, C2, C3, C4, C5



Digraph representation

The adjacency matrix would look like this:

	C_1	C_2	C_3	C_4	C_5
C_1	0	0	1	0	0
C_2	0	0	1	0	0
C_3	0	0	0	1	1
C_4	0	0	0	0	1
C_5	0	0	0	0	0

Decrease by Constant (or Decrease by One)

Topological Sorting

Source-removal algorithm:

Steps involved in finding the topological ordering of a DAG:

Step-1: Compute in-degree (number of incoming edges) for each of the vertex present in the DAG and initialize the count of visited nodes as 0.

Step-2: Pick all the vertices with in-degree as 0 and add them into a queue (Enqueue operation)

Step-3: Remove a vertex from the queue (Dequeue operation) and then.

a. Increment count of visited nodes by 1.

b. Decrease in-degree by 1 for all its neighboring nodes.

c. If in-degree of a neighboring nodes is reduced to zero, then add it to the queue.

Step 5: Repeat Step 3 until the queue is empty.

Step 5: If count of visited nodes is **not** equal to the number of nodes in the graph then the topological sort is not possible for the given graph.

How to find in-degree of each node?

Traverse the array of edges and simply increase the counter of the destination node by 1

Traverse the list for every node and then increment the in-degree of all the nodes connected to it by 1

$O(V+E)$

Decrease by Constant (or Decrease by One)

Topological Sorting

- **Source-removal algorithm:**

- **Step1: Initialize the In-degree Array:** Create an array `in_degree` to store the number of prerequisites for each course.

Course	C_1	C_2	C_3	C_4	C_5
In-degree	0	0	2	1	2

- **Step2: Find sources and add to the Queue:** Identify all courses with an `in-degree` of 0. These are our initial sources.

- **Step3: Process the Queue:** Repeatedly take a course from the front of the queue, add it to our final sorted list, and update the in-degrees of the courses that depend on it.

- **Step 4: When the queue is empty.** The algorithm terminates.

Final Result: A valid order to take the courses is $[C_1, C_2, C_3, C_4, C_5]$

Decrease by Constant (or Decrease by One)

Generating Permutations

Generating all $n!$ permutations of $\{1, \dots, n\}$?

Generate all $(n-1)!$ permutations of $\{1, \dots, n-1\}$?

insert n in each of the n possible positions among elements of every permutation of $n - 1$ elements

Minimal-change Algorithm

If $n = 1$ return 1; otherwise, generate recursively the list of all permutations of $1, 2, \dots, n-1$ and then insert n into each of those permutations by starting with inserting n into $1, 2, \dots, n-1$ by moving right to left and then switching direction for each new permutation.

Example: $n=3$

start	1
insert 2 into 1 right to left	12 21
insert 3 into 12 right to left	123 132 312
insert 3 into 21 left to right	321 231 213

The traveling salesman problem

we can compute the length of a new tour from the length of its predecessor in constant rather than linear time

algorithm's speed

Benefits

minimal-change??

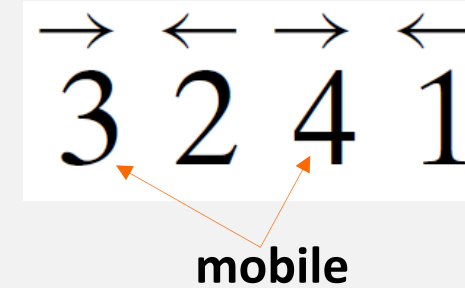
each permutation can be obtained from its immediate predecessor by exchanging just two elements in it.

Decrease by Constant (or Decrease by One)

Generating Permutations

get the same ordering of permutations of n elements without explicitly generating permutations for smaller values of n

- associate a direction with each element k in a permutation
- The element k is said to be **mobile** in such an arrow-marked permutation if its arrow points to a smaller number adjacent to it



Try it

ALGORITHM *JohnsonTrotter(n)*

//Implements Johnson-Trotter algorithm for generating permutations

//Input: A positive integer n

//Output: A list of all permutations of $\{1, \dots, n\}$

initialize the first permutation with $\overleftarrow{1} \overleftarrow{2} \dots \overleftarrow{n}$

while the last permutation has a mobile element **do**

 find its largest mobile element k

 swap k with the adjacent element k 's arrow points to

 reverse the direction of all the elements that are larger than k

 add the new permutation to the list

List	k	Swap	Element > k	Reverse
$\overleftarrow{1} \overleftarrow{2} \overleftarrow{3}$	3	2&3	None	-
$\overleftarrow{1} \overleftarrow{3} \overleftarrow{2}$	3	1&3	None	-
$\overleftarrow{3} \overleftarrow{1} \overleftarrow{2}$	2	1&2	3	$\overrightarrow{3}$
$\overrightarrow{3} \overleftarrow{2} \overleftarrow{1}$	3	3&2	None	-
$\overrightarrow{2} \overrightarrow{3} \overleftarrow{1}$	3	3&1	None	-
$\overrightarrow{2} \overleftarrow{1} \overrightarrow{3}$				

Decrease by Constant (or Decrease by One)

Generating Permutations

Generate permutations in increasing order??

For example, for $n = 3$

123 132 213 231 312 321

ALGORITHM *LexicographicPermute(n)*

//Generates permutations in lexicographic order

//Input: A positive integer n

//Output: A list of all permutations of $\{1, \dots, n\}$ in lexicographic order

initialize the first permutation with $12 \dots n$

while last permutation has two consecutive elements in increasing order **do**

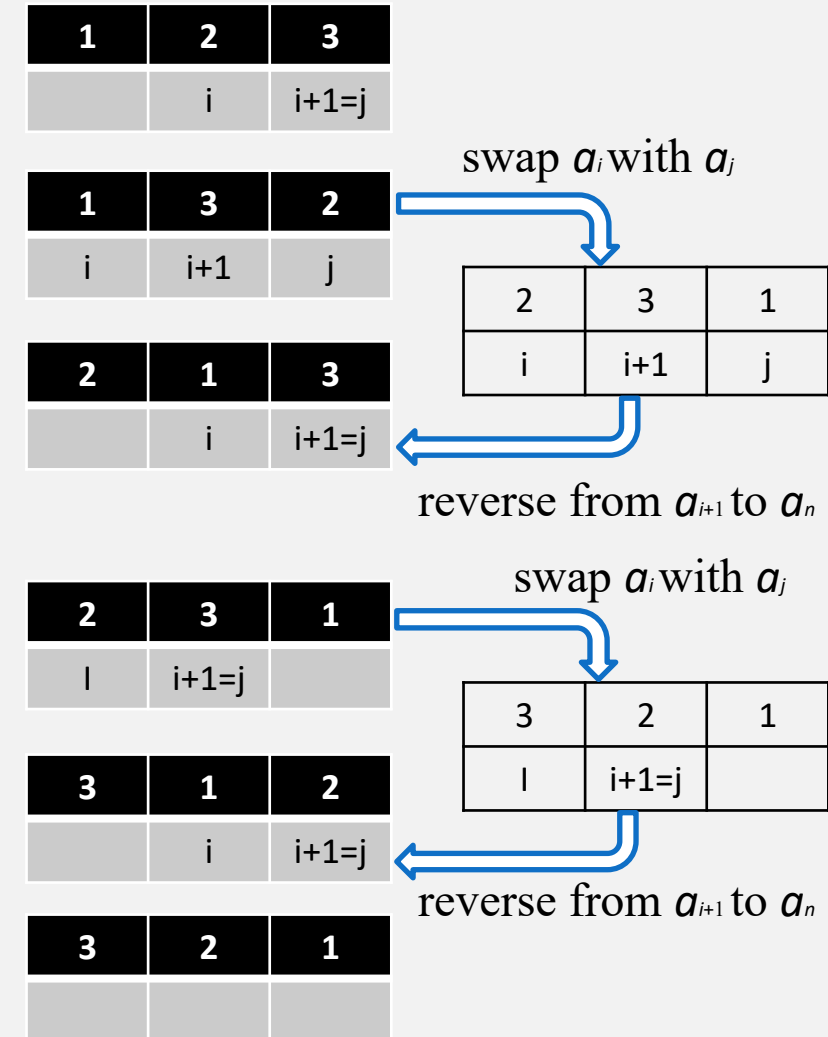
let i be its largest index such that $a_i < a_{i+1}$ // $a_{i+1} > a_{i+2} > \dots > a_n$

find the largest index j such that $a_i < a_j$ // $j \geq i + 1$ since $a_i < a_{i+1}$

swap a_i with a_j // $a_{i+1}a_{i+2} \dots a_n$ will remain in decreasing order

reverse the order of the elements from a_{i+1} to a_n inclusive

add the new permutation to the list



Decrease by Constant (or Decrease by One)

Generating Subsets

Generating subsets bottom up

Generate all 2^n subsets of the set $A = \{a_1, a_2, \dots, a_n\}$

Not containing a_n : all subsets of $\{a_1, a_2, \dots, a_{n-1}\}$

Containing a_n : Add a_n to each subset of $\{a_1, a_2, \dots, a_{n-1}\}$

n	subsets							
0	$\emptyset$							
1	$\emptyset$	$\{a_1\}$						
2	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$				
3	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$	$\{a_3\}$	$\{a_1, a_3\}$	$\{a_2, a_3\}$	$\{a_1, a_2, a_3\}$

Not containing a_3 Containing a_3

Decrease by Constant (or Decrease by One)

Generating Subsets

one-to-one correspondence between all 2^n subsets of an n element set $A = \{a_1, a_2, \dots, a_n\}$ and all 2^n bit strings $b_1 b_2 \dots b_n$ of length n .

Example for $n = 3$

generating successive binary numbers from 0 to $2^n - 1$



	b_1	b_2	b_3					
bit strings	000	001	010	011	100	101	110	111
subsets	$\emptyset$	$\{a_3\}$	$\{a_2\}$	$\{a_2, a_3\}$	$\{a_1\}$	$\{a_1, a_3\}$	$\{a_1, a_2\}$	$\{a_1, a_2, a_3\}$

$$b_i = \begin{cases} 1 & \text{if } a_i \text{ belongs to the subset} \\ 0 & \text{otherwise} \end{cases}$$

Decrease by Constant (or Decrease by One)

Generating Subsets

Binary reflected Gray code: minimal-change algorithm for generating 2^n bit strings corresponding to all the subsets of an n -element set where $n > 0$

If $n=1$ make list L of two bit strings 0 and 1

else

generate recursively list $L1$ of bit strings of length $n-1$

copy list $L1$ in reverse order to get list $L2$

add 0 in front of each bit string in list $L1$

add 1 in front of each bit string in list $L2$

append $L2$ to $L1$ to get L

return L

Following is 2-bit sequence ($n = 2$)

00 01 11 10

Following is 3-bit sequence ($n = 3$)

000 001 011 010 110 111 101 100

And Following is 4-bit sequence ($n = 4$)

0000 0001 0011 0010 0110 0111 0101 0100

1100 1101 1111 1110 1010 1011 1001 1000

The Knapsack problem

Benefit

we can compute the total weight of a new solution from the weight of its predecessor in constant rather than linear time

Decrease by Constant (or Decrease by One)

Generating Subsets (using BRGC algorithm)

Knapsack Problem Example (3 Items)

- **Knapsack Capacity:** 5 kg
- **Items Available:**
 - Item A: Weight = 2 kg, Value = \$60
 - Item B: Weight = 3 kg, Value = \$80
 - Item C: Weight = 4 kg, Value = \$90

The goal is to find the combination of items with the highest total value that does not exceed the 5 kg weight limit. The standard approach is to check all $2^3 = 8$ possible subsets.

Decrease by Constant (or Decrease by One)

Generating Subsets (using BRGC algorithm)

BRGC Sequence	Bit Change	Items in Knapsack	Total Weight (kg)	Total Value (\$)	Valid? (Weight $\leq$ 5 kg)
000	-	{}	0	0	✓
001	+C	{C}	4	90	✓
011	+B	{C, B}	4+3 = 7	90+80=170	✗ (Too heavy)
010	-C	{B}	3	80	✓
110	+A	{B, A}	3+2 = 5	80+60=140	✓
111	+C	{B, A, C}	5+4 = 9	140+90=230	✗ (Too heavy)
101	-B	{A, C}	2+4 = 6	60+90=150	✗ (Too heavy)
100	-C	{A}	2	60	✓

The Optimal Solution

By checking all valid combinations, we can see the maximum value.

- {C}: Value = \$90
- {B}: Value = \$80
- {B, A}: Value = \$140
- {A}: Value = \$60

The highest total value is **\$140** by taking **Items A and B**. This example shows how BRGC allows for a quick, incremental check of all subsets to find the optimal solution.

Decrease-by-Constant-Factor

Fake-Coin Problem

There are n identically looking coins one of which is fake.
There is a balance scale but there are no weights; the scale can tell whether two sets of coins weigh the same and, if not, which of the two sets is heavier (but not by how much). Design an efficient algorithm for detecting the fake coin. Assume that the fake coin is known to be lighter than the genuine ones.

Decrease by factor 2 algorithm

$$W(n) = W(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1, \quad W(1) = 0$$

$$W(n) = \lfloor \log_2 n \rfloor$$

Decrease by factor 3 algorithm

$$\log_3 n$$

Decrease-by-Constant-Factor

Russian peasant method

The problem: Compute the product of two positive integers

Can be solved by a decrease-by-half algorithm based on the following formulas.

For even values of n :

$$n \cdot m = \frac{n}{2} \cdot 2m$$

For odd values of n :

$$n \cdot m = \frac{n-1}{2} \cdot 2m + m$$

Compute $20 * 26$

n	m	
20	26	
10	52	
5	104	
2	208	+104
1	416	<u>416</u>
		520

Decrease-by-Constant-Factor

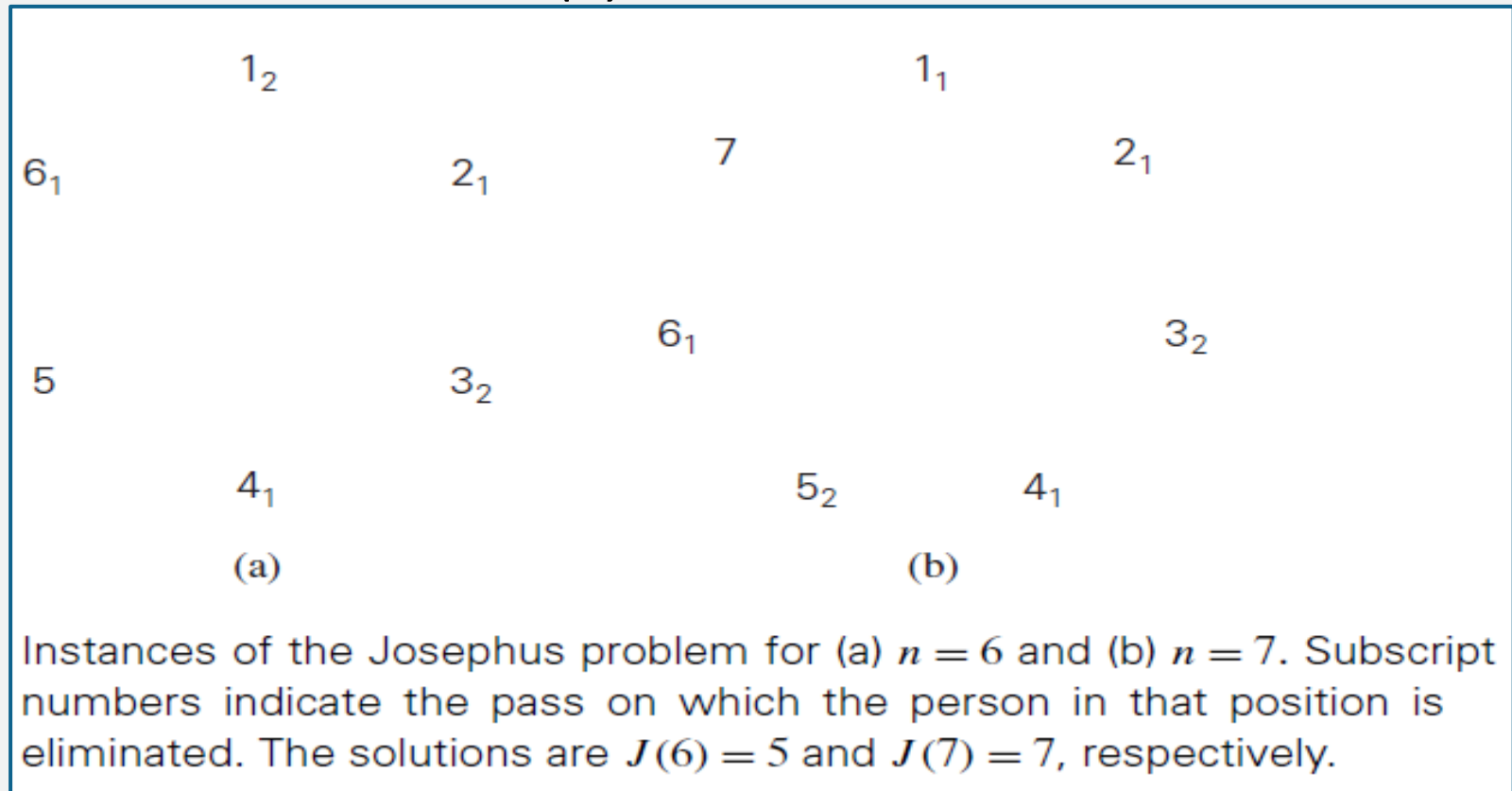
Josephus problem

let n people numbered 1 to n stand in a circle. Starting the grim count with person number 1, we eliminate every second person until only one survivor is left.

The problem is to determine the survivor's number $J(n)$.

$$J(2k) = 2J(k) - 1$$

$$J(2k + 1) = 2J(k) + 1$$



Variable Size Decrease

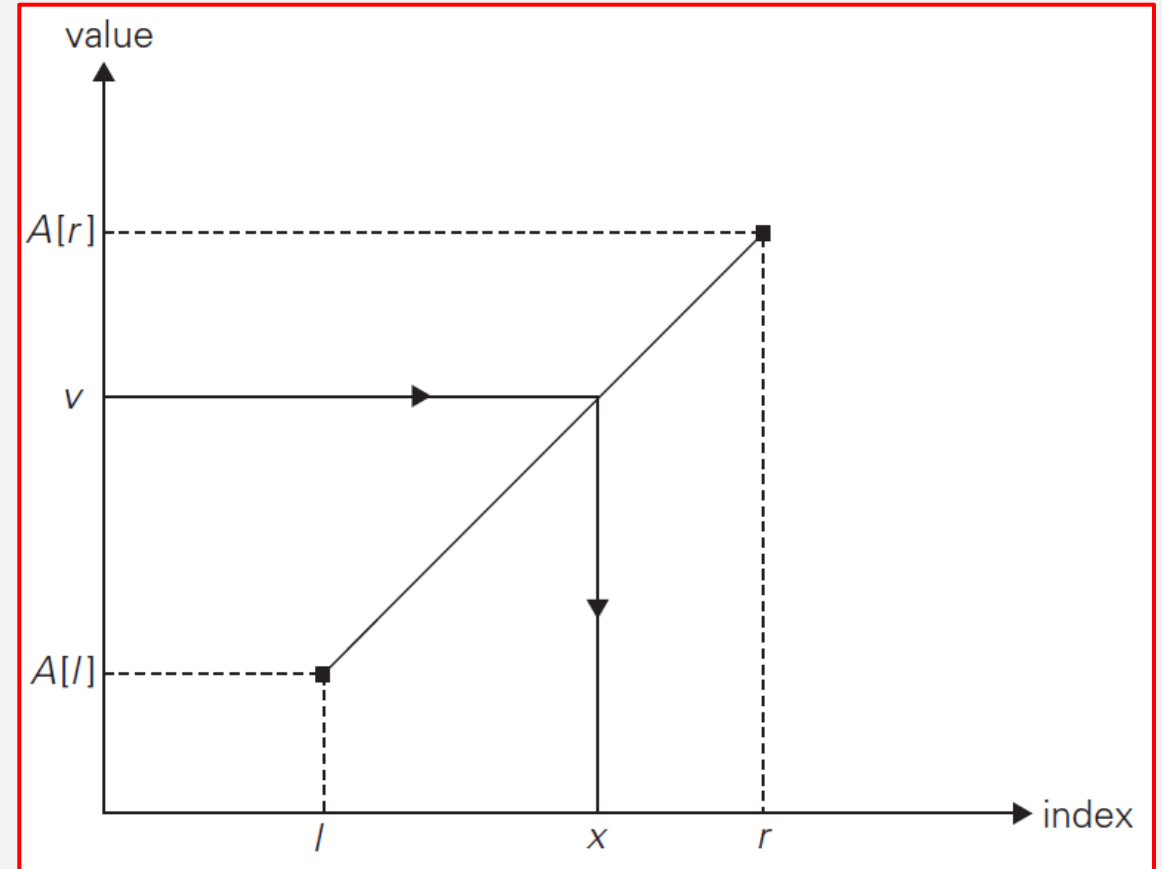
Interpolation Search

- The values in a sorted array are uniformly distributed.
- Unlike binary search which goes to the middle element to check, we may go to different locations according to the value of the key being searched
- If the value of the key is closer to the last element, interpolation search is likely to start search toward the end side.

Variable Size Decrease

Interpolation Search

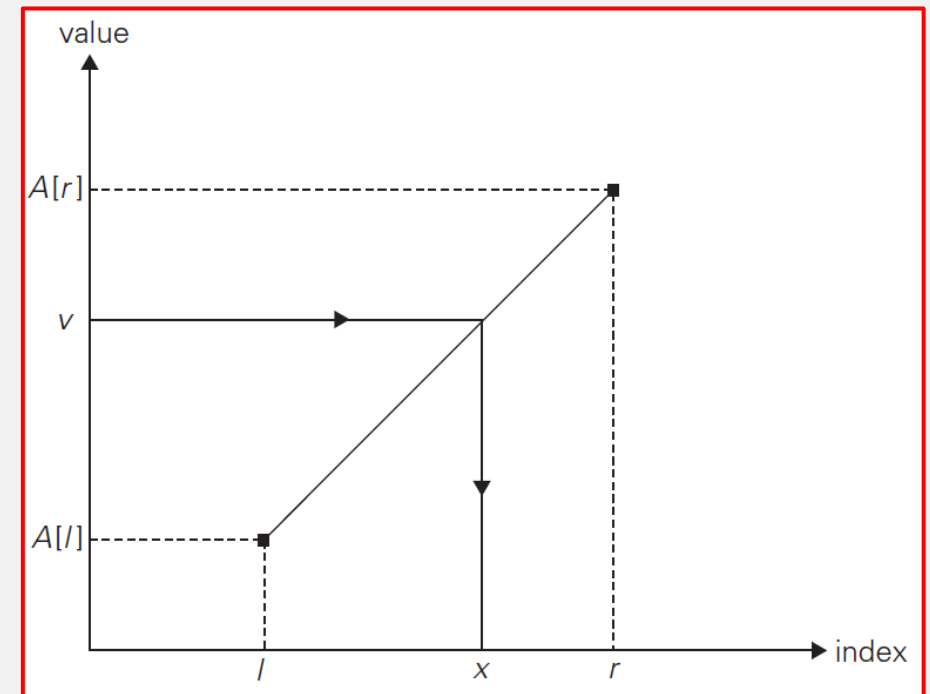
Searches a sorted array similar to binary search but estimates location of the search key in $A[l..r]$ by using its value v . Specifically, the values of the array's elements are assumed to grow linearly from $A[l]$ to $A[r]$ and the location of v is estimated as the x -coordinate of the point on the straight line through $(l, A[l])$ and $(r, A[r])$ whose y -coordinate is v :



Variable Size Decrease

Interpolation Search

```
static int interpolationSearch(int v, int A[]) {  
    int l = 0, r = (A.length - 1);  
    while (l <= r && v >= A[l] && v <= A[r]){  
        if (l == r){  
            if (A[l] == v)  
                return l;  
            return -1;  
        }  
        int pos = l + (((r-l) / (A[r]-A[l]))*(v - A[l]));  
        if (A[pos] == v)  
            return pos;  
        if (A[pos] < v) l = pos + 1;  
        else r = pos - 1;  
    }  
    return -1;  
}
```



$$x = l + \left\lfloor \frac{(v - A[l])(r - l)}{A[r] - A[l]} \right\rfloor$$

Variable Size Decrease Interpolation Search

$$x = l + \left\lfloor \frac{(v - A[l])(r - l)}{A[r] - A[l]} \right\rfloor$$

18 ?

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	12	13	16	18	19	20	21	22	23	24	33	35	42	47

Step 1 : $p = \frac{14}{2} = 7$ $A[p] = 21 \geq 18$

0	1	2	3	4	5	6
10	12	13	16	18	19	20

Step 2 : $p = \frac{6}{2} = 3$ $A[p] = 16 \leq 18$

4	5	6
18	19	20

Step 3 : $p = \frac{6+4}{2} = 5$ $A[p] = 19 \geq 18$

4
18

Step 4 : $p = \frac{4+4}{2} = 4$ $A[p] = 18$

Binary Search

Step 1 : $p = 0 + \frac{14}{37} \times (18 - 10) = 3$ $A[p] = 16 \leq 18$

4	5	6	7	8	9	10	11	12	13	14
18	19	20	21	22	23	24	33	35	42	47

Step 2 : $p = 4 + \frac{10}{29} \times (18 - 18) = 4$ $A[p] = 18 = 18$

4
18

Interpolation Search

Time Complexity: If elements are uniformly distributed, then $O(\log \log n)$. In worst case it can take upto $O(n)$

Variable Size Decrease

Analysis of Interpolation Search

- Preferable to binary search only for VERY large arrays and/or expensive comparisons
- Has a counterpart, the method of false position (regula falsi), for solving equations in one unknown.

Variable-Size-Decrease

Partitioning Algorithms

The *selection problem* is the problem of finding the k^{th} smallest element in a list of n numbers

for $k = 1$ or $k = n$, scan the list to find the smallest or largest element

for $k = n/2$, find an element that is not larger than one half of the list's elements and not smaller than the other half. This middle value is called the *median*

Solution 1

Sort the list first and then select the k^{th} element in the output of a sorting algorithm. the algorithm's efficiency is in $O(n \log n)$ when using mergesort. (The sorting-based Algorithms)

Why sorting the entire list ??! We need only to find the k^{th} smallest element.

Variable-Size-Decrease

Partitioning Algorithms

The *selection problem* is the problem of finding the k^{th} smallest element in a list of n numbers

- median: $k = \lceil n/2 \rceil$

Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median = ?

The median is used in statistics as a measure of an average value of a sample. In fact, it is a better (more robust) indicator than the mean, which is used for the same purpose.

- Example: Post Office Location Problem

Given n village locations along a straight highway, where should a new post office be located to minimize the average distance from the villages to the post office?

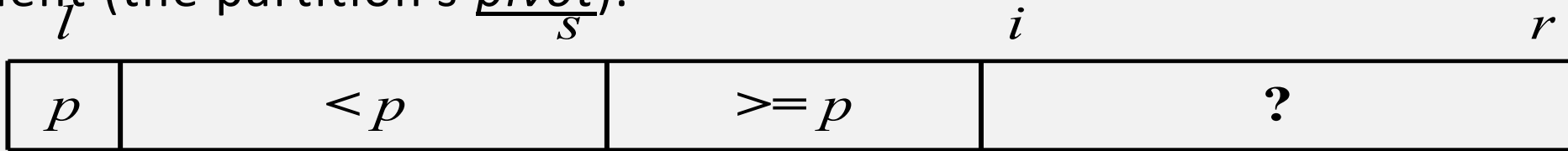
Two Partitioning Algorithms

There are two principal ways to partition an array:

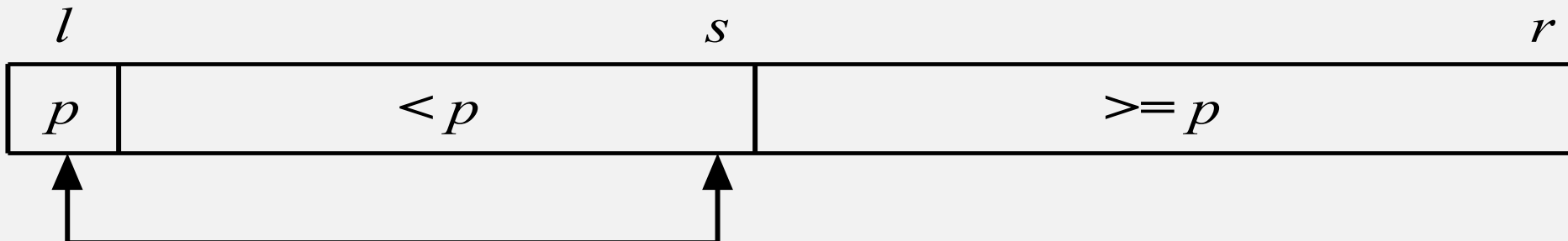
- One-directional scan (Lomuto's partitioning algorithm)
- Two-directional scan (Hoare's partitioning algorithm)

Variable-Size-Decrease Partitioning Algorithm

Scans the array left to right maintaining the array's partition into three contiguous sections: $< p$, $\geq p$, and unknown, where p is the value of the first element (the partition's pivot).



On each iteration the unknown section is decreased by one element until it's empty and a partition is achieved by exchanging the pivot with the element in the split position s .



Variable-Size-Decrease Partitioning Algorithms

Partitioning a given list around some value p (the first) so that the left part contains all the elements $\leq$ to p , followed by the **pivot** p itself, followed by all the elements $\geq$ to p .

ALGORITHM *LomutoPartition*($A[l..r]$)

//Using first element as pivot

//Input: A subarray $A[l..r]$ of array $A[0..n-1]$

//Output: Partition of $A[l..r]$ and the new position of the pivot

$p \leftarrow A[l]$

$s \leftarrow l$

for $i \leftarrow l+1$ **to** r **do**

if $A[i] < p$

$s \leftarrow s + 1$;

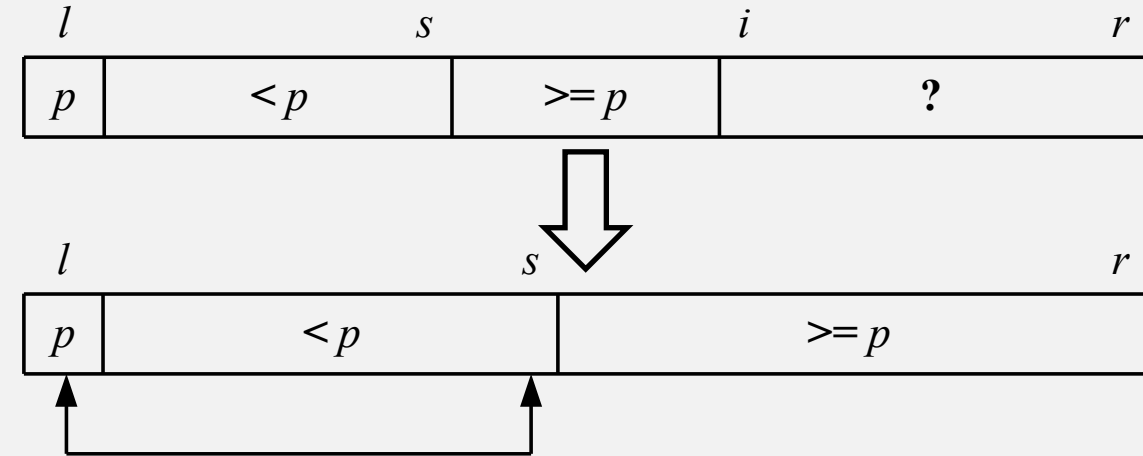
 swap($A[s]$, $A[i]$)

endif

endfor

swap($A[l]$, $A[s]$)

return s



Quickselect Algorithm (finding the k^{th} smallest element)

If $s = k-1$, the problem is solved;

if $s > k-1$, look for the k -th smallest element in the left part.

if $s < k-1$, look for the $(k-s)$ -th smallest element in the right part.

continue until $s = k-1$

A faster algorithm is usually based on the array partitioning.

Variable-Size-Decrease Partitioning Algorithms

recursive version

ALGORITHM *Quickselect*($A[l..r]$, k)

//Solves the selection problem by recursive partition-based algorithm

//Input: Subarray $A[l..r]$ of array $A[0..n - 1]$ of orderable elements and

// integer k ($1 \leq k \leq r - l + 1$)

//Output: The value of the k th smallest element in $A[l..r]$

$s \leftarrow \text{LomutoPartition}(A[l..r])$ **if** $s = k - 1$ **return** $A[s]$

else if $s > l + k - 1$ *Quickselect*($A[l..s - 1]$, k)

else *Quickselect*($A[s + 1..r]$, $k - 1 - s$)

For the non-recursive version, we need not even adjust the value of k but just continue until $s = k - 1$.

- The case of $k = n$ and a strictly increasing array

$$\begin{aligned} C_{\text{worst}}(n) &= (n - 1) + (n - 2) + \dots + 1 \\ &= (n - 1)n / 2 \in \Theta(n^2) \end{aligned}$$

- The split solves the selection problem

$$C_{\text{best}}(n) = (n - 1) \in \Theta(n)$$

Variable-Size-Decrease

Example: Lomuto Partitioning with First Element as Pivot

Array = [8, 3, 1, 7, 0, 10, 2]

Pivot = first element = 8

- Initial: [8, 3, 1, 7, 0, 10, 2] (pivot=8, i=0)
- Compare $3 < 8 \rightarrow$ swap with A[1] $\rightarrow$ [8, 3, 1, 7, 0, 10, 2] (no change, i=1)
- Compare $1 < 8 \rightarrow$ swap with A[2] $\rightarrow$ [8, 3, 1, 7, 0, 10, 2] (i=2)
- Compare $7 < 8 \rightarrow$ swap with A[3] $\rightarrow$ [8, 3, 1, 7, 0, 10, 2] (i=3)
- Compare $0 < 8 \rightarrow$ swap with A[4] $\rightarrow$ [8, 3, 1, 7, 0, 10, 2] (i=4)
- Compare $10 < 8?$ ✗ skip
- Compare $2 < 8 \rightarrow$ swap with A[5] $\rightarrow$ [8, 3, 1, 7, 0, 2, 10] (i=5)
- Finally swap pivot with A[i] $\rightarrow$ [2, 3, 1, 7, 0, 8, 10]

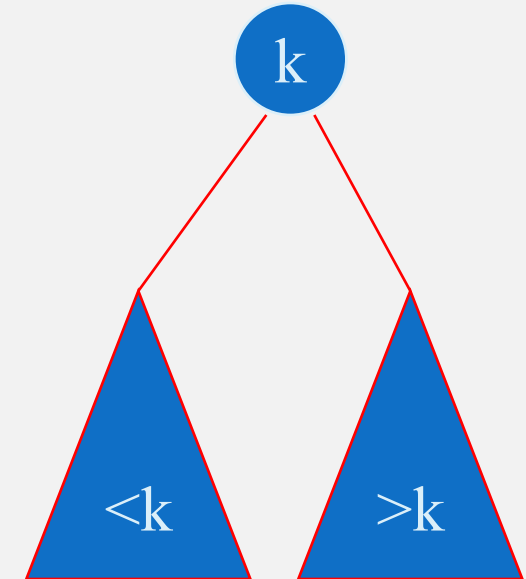
Variable-Size-Decrease

Searching in Binary Search Tree

Several algorithms on BST requires recursive processing of just one of its subtrees:

- Searching
- Insertion of a new key
- Finding the smallest (or the largest) key

```
// A utility function to search a given key in BST
public Node search(Node root, int key)
{ // Base Cases: root is null or key is present at root
  if (root==null || root.key==key)
    return root;
  // val is greater than root's key
  if (root.key > key)
    return search(root.left, key);
  // val is less than root's key
  return search(root.right, key);
}
```



Efficiency

worst case: $C(n) = n$

average case: $C(n) \approx 2 \ln n \approx 1.39 \log_2 n$

Searching in Binary Search Tree

Algorithm $BTS(x, v)$

//Searches for node with key equal to v in BST rooted at node x

if $x = \text{NIL}$ return -1

else if $v = K(x)$ return x

else if $v < K(x)$ return $BTS(\text{left}(x), v)$

else return $BTS(\text{right}(x), v)$